

ReMax：一种用于对齐大语言模型的简单、有效且高效的强化学习方法

论文中文翻译稿

根据用户提供论文 *ReMax: A Simple, Effective, and Efficient Reinforcement Learning Method for Aligning L*

源文件：ReMax.pdf

本文依据用户上传论文内容整理翻译，公式沿用原文记号，参考文献条目保留英文原始信息。

摘要

基于人类反馈的强化学习（RLHF）是对齐大语言模型（LLM）的关键技术，通常与近端策略优化（PPO）算法配套使用。尽管 PPO 是为一般强化学习任务设计的一种强大方法，但对于 LLM 来说它显得过于复杂，从而带来了繁琐的超参数调节和显著的计算负担。为提升 RLHF 的效率，本文提出 ReMax。该方法利用了 RLHF 的三个性质：仿真速度快、状态转移是确定性的，以及奖励是在整条轨迹结束后才给出的。PPO 并未利用这些性质，因此并不十分适合 RLHF。

ReMax 建立在经典的 REINFORCE 算法之上，不再像 PPO 那样需要额外训练一个价值模型，并进一步引入了一种新的方差缩减技术。与 PPO 相比，ReMax 具有若干优势：实现更简单，能够消除 PPO 中四个以上的超参数，降低 GPU 显存占用，并缩短训练时间。在训练 7B 模型时，ReMax 相比 PPO 大约可节省 46% 的 GPU 显存，并且能够在 A800-80GB GPU 上训练，而 PPO 则需要借助显存节省型的 offloading 技术。将 ReMax 应用于 Mistral-7B 模型后，在 AlpacaEval 榜单上取得了 94.78% 的胜率，在 MT-bench 上获得 7.739 分，为开源 7B 模型树立了新的 SOTA。实验结果表明，ReMax 在克服 PPO 于 LLM 场景下局限性的同时，也保持了很强的有效性。

1 引言

对齐是让大语言模型遵循人类偏好的关键。当前常见的两种路线分别是指令微调和基于人类反馈的强化学习。前者依赖大量预先准备好的人类标注数据，因此需要研究者在训练前就明确模型应该表现出什么行为，这一过程通常较为费力。相比之下，RLHF 不要求事先把所有行为都定义完整。简单来说，RLHF 将文本生成任务改写为一个序列决策问题，并利用强化学习算法求解。GPT-4 技术报告也强调了 RLHF 的重要性。

尽管 RLHF 非常成功，但其广泛应用仍受到巨大计算开销的限制。论文将 RLHF 归纳为三个主要步骤：第一步是对 LLM 进行监督微调（SFT），以获得一个较好的初始化；第二步是根据人类偏好数据训练奖励模型（RM）；第三步是使用强化学习算法，在仅含 prompt 的数据集上继续微调 LLM。

困难主要集中在第三步。此时模型需要不断生成并改进回答，以获得更高奖励。当前这一阶段最常用的算法是 PPO，相关工作与开源系统大多沿用了这一选择。然而，PPO 因为要引入额外的价值模型及配套训练组件，显著增加了计算需求。在实践中，使用 PPO 进行训练所花费的时间可能比

前两个步骤加起来还要长四倍，GPU 显存消耗至少翻倍，这使得在计算资源受限时，RLHF 很难真正落地。

针对这一现象，本文聚焦 RLHF 的第三步，目标是让它更容易使用、也更便宜。作者注意到，已有研究曾尝试通过 gradient checkpointing、ZeRO、offload 以及混合训练引擎等工程技术来缓解这一问题，但这些技术通常会拖慢计算速度。即便如此，PPO 过重的显存占用依然是一个根本挑战。

1.1 本文贡献

本文重新审视了 RLHF 的问题形式，希望据此设计新的算法。作者指出，RLHF 与一般强化学习任务相比，存在三个非常重要的差异性性质：仿真速度快、状态转移确定、奖励位于整条轨迹层面。所谓仿真速度快，是指一条轨迹，也就是 LLM 的完整回答，可以以很小的时间开销快速生成出来；所谓状态转移确定，是指当前上下文只由历史 token 和当前新生成的 token 决定；所谓轨迹级奖励，则是指奖励模型只会在整个回答完成之后给出一个标量分数。这三个性质在 PPO 中并没有被充分利用，因此作者认为，PPO 不是最适合 LLM 场景下 RLHF 的方法。

基于这些观察，本文提出了一个专门针对 RLHF 的新算法 ReMax。ReMax 建立在经典的 REINFORCE 之上，其核心训练逻辑可以概括为：对每个 prompt，先从模型中随机采样一个回答，再贪心生成一个回答，用奖励模型分别打分后做差，最后将该差值作为权重乘到采样回答的对数概率和上，并最小化对应损失。与 PPO 一样，这种做法仍是无偏的随机策略梯度估计；但不同于 PPO，ReMax 不再依赖价值模型。由于 PPO 中的价值模型既要占显存，又要参与反向传播和优化器状态维护，因此它往往会显著拉高训练时间和显存占用。为在不引入价值模型的前提下保持训练稳定，ReMax 进一步设计了一种新的方差缩减机制。

作者总结了 ReMax 相对于 PPO 的几项优势。第一，ReMax 更简单，实现只需大约 6 行主逻辑代码，而 PPO 往往需要 30 多行，并且 ReMax 去掉了 PPO 中至少 4 个需要精细调节的关键超参数，例如重要性采样裁剪系数、GAE 系数、价值模型学习率以及离策略训练轮数。对于大模型来说，超参数搜索本身就是昂贵的，因此这一点非常关键。第二，ReMax 显著节省显存，使得在资源有限的环境中也能进行 RLHF 训练，而 PPO 往往会直接耗尽显存。例如，PPO 在不使用 optimizer offloading 的情况下无法在 A800-80GB 上训练 Llama-2-7B，而 ReMax 可以做到。第三，ReMax 计算更高效，因为它省去了价值模型训练，同时节省下来的显存还能用于扩大 batch size，从而提升吞吐率。实验中它大约比 PPO 快 1.6 倍。第四，理论分析证明了 ReMax 具有收敛性和方差缩减性质，实证结果也表明它至少能达到、甚至超过 PPO 的效果。基于 Mistral-7B 的最佳模型在 AlpacaEval 上取得 94.78% 的胜率，在 MT-bench 上达到 7.739，刷新了开源 7B 模型的最好成绩。

作者还指出，ReMax 在计算效率上也能与无奖励模型方法，如 DPO，相媲美，但性能更优。其代码已公开在 GitHub 上。

2 相关工作

本文简要回顾 RLHF 与 LLM 相关研究，并在附录中给出更详细讨论。RLHF 已在诸多工作中被系统研究，包括 Anthropic、OpenAI 等路线的代表性方法。近年来，也有很多研究专门分析 RLHF 的尺度规律、样本复杂度以及优化误差。

在 RLHF 的第三步优化方面，已有研究从不同角度尝试改进。例如，有工作为 NLP 任务中的 RL 提供基准与基线，也有工作研究如何在 PPO 中高效地使用 LoRA，或者系统总结 PPO 在 RLHF 实现中的诸多工程细节。除 RL 路线之外，DPO 则尝试绕过显式奖励模型训练，直接从偏好数据中学习策略。

与这些工作相比，本文的不同点在于，它是从 RLHF 任务本身的特殊性质出发重新设计优化算法，而不是仅在 PPO 上做工程级改造。作者同时承认，PPO 在很多公开实验中通常优于其他替代方法，因此也长期受到开源工具链的偏爱。

3 问题形式化

将一个大语言模型记作 π_θ ，其中参数 $\theta \in \mathbb{R}^d$ 。模型从有限词表 $\mathcal{V} = \{1, \dots, |\mathcal{V}|\}$ 中逐步生成 token。给定上下文 $(a_1, \dots, a_t)$ ，模型按分布 $a_{t+1} \sim \pi_\theta(\cdot | a_1, \dots, a_t)$ 采样下一个 token。这个过程持续进行，直到遇到 EOS token，或者达到最大长度 T 。为简化记号，原文假设回答长度固定为 T ，若提前结束则在 EOS 后做 padding。

虽然 LLM 经过了大规模预训练，能够完成 next-token prediction，但它们往往难以自然地产生符合人类偏好的回答，例如更好地遵循指令、输出更像人类写作的文本。为此，实践中普遍采用 RLHF 进行进一步微调。

在 RLHF 设定中，奖励模型 r 会对给定 prompt x 及完整回答 $(a_1, \dots, a_T)$ 输出一个标量 $r(x, a_{1:T})$ ，通常分数越高越好。该奖励模型一般通过人类偏好数据上的二元分类目标训练得到：

$$\max_r \mathbb{E}_{(x, a_{1:T}, a'_{1:T}) \sim D} [\log \sigma(r(x, a_{1:T}) - r(x, a'_{1:T}))].$$

其中 D 是偏好数据集， $a_{1:T}$ 表示偏好更高的回答， $a'_{1:T}$ 表示相应的劣回答， σ 是 sigmoid 函数。实践中，这个奖励模型通常由 LLM 参数初始化。

随后，LLM 的目标是最大化奖励：

$$\max_{\theta} \mathbb{E}_{x \sim \rho} \mathbb{E}_{a_{1:T} \sim \pi_\theta} [r(x, a_{1:T})]. \quad (1)$$

其中 ρ 表示 prompt 分布，既可以包含偏好数据中的训练 prompt，也可以包含没有偏好标注的新 prompt。为简化叙述，正文先略去与参考模型相关的 KL 正则项，附录 B 再进行说明。

3.1 用强化学习求解奖励最大化

作者首先把式 (1) 写成一个强化学习中的序列决策问题。这一设定也是很多已有 RLHF 工作采用的方式。对应的马尔可夫决策过程记作 $M = (\mathcal{S}, \mathcal{A}, P, r, \rho, T)$ ，其中状态空间和动作空间分别为 $\mathcal{S}$ 和 $\mathcal{A}$ ， P 是状态转移函数， r 是一步奖励函数， ρ 是初始状态分布，轨迹长度为 T 。在 MDP 框架下，目标是最大化长期回报，即逐步奖励的累加：

$$\max_{\theta} \mathbb{E}_{s_1 \sim \rho} \mathbb{E}_{a_{1:T} \sim \pi_\theta} \left[\sum_{t=1}^T r(s_t, a_t) \mid s_{t+1} \sim P(\cdot | s_t, a_t) \right]. \quad (2)$$

在 LLM 场景中，状态 s_t 可以表示为先前已经生成的 token 序列，即 $s_t = (x, a_1, \dots, a_{t-1})$ ；动作 a_t 则是在词表中选择下一个 token；初始状态 s_1 就是 prompt x ，因此 ρ 对应 prompt 分布。

由于奖励模型只对完整回答打分，而不对中间 token 打分，作者将其改写为：中间时刻奖励均为 0，仅在最后一步给出最终回答的奖励，即

$$r(s_t, a_t) = \begin{cases} 0, & t \neq T, \\ r(x, a_{1:T}), & t = T. \end{cases} \quad (3)$$

状态转移函数则只是把当前 token 追加到历史中，因此有

$$P(s_{t+1}|s_t, a_t) = \begin{cases} 1, & s_{t+1} = (*s_t, a_t), \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

在这样的建模下，任何能求解式 (2) 的 RL 算法，都可以等价地用来求解式 (1)。

3.2 问题 (2) 的自然解法：PPO

PPO 是解决一般强化学习问题的一个非常有效的算法。本文并不展开所有实现细节，只给出足以支撑后续讨论的简化描述。PPO 通过最大化一个近端代理目标进行训练：

$$L_{\text{sur}}(\theta) = \mathbb{E}_{x \sim \rho} \mathbb{E}_{a_{1:T} \sim \pi_{\theta_{\text{old}}}} \sum_{t=1}^T A(s_t, a_t) \min\{\psi(s_t, a_t), \text{clip}(\psi(s_t, a_t), 1 - \delta, 1 + \delta)\}. \quad (5)$$

这里轨迹由旧策略 $\pi_{\theta_{\text{old}}}$ 采样得到，再通过重要性采样比值

$$\psi(s_t, a_t) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

来降低偏差； $\delta > 0$ 是裁剪超参数； $A(s_t, a_t)$ 是优势函数，通常由一步奖励和价值模型 V 组合估计，例如

$$A(s_t, a_t) \approx r(s_t, a_t) + V(s_{t+1}) - V(s_t).$$

价值模型 $V(s)$ 通过 TD 学习来逼近从任一状态出发的长期回报。此外，PPO 往往还要借助 GAE 计算优势，这又引入了一个额外需要调节的超参数。简言之，PPO 为优化引入了一个可训练价值模型以及一整套与之配套的超参数和工程组件。

3.3 为什么 PPO 并不最适合 RLHF

作者指出，在 RLHF 中，PPO 的价值模型会显著增加计算负担，主要原因有两点。第一，价值模型通常和语言模型规模相当；第二，它需要参与训练，因此还会带来梯度、激活和优化器状态等额外开销。结果是，价值模型在训练时的显存占用可能是仅做推理时的四倍以上。对于 7B 模型而言，PPO 总显存中大约有一半都用于训练价值模型。因此，如果能让 RLHF 更高效，最关键的就是想办法去掉或替代价值模型。

进一步地，作者回到理论层面重新审视价值模型存在的原因。按照 MDP 理论，长期期望回报要通过优势函数参与策略梯度估计，而价值模型正是为了学习这种期望回报。当环境转移是随机的时，价值模型可以帮助更充分地利用过去经验；当仿真或 rollout 很慢时，价值模型还可以快速估计长期回报。但对于 LLM 中的 RLHF，这两点动机都不成立。

作者由此总结出 RLHF 的三个关键性质。其一，仿真速度快。传统 RL 任务中得到长期回报往往很昂贵，例如星际争霸、Atari 或机器人控制都需要耗费大量环境交互时间；而在 RLHF 中，长期回报本质上就是让 LLM 生成一个回答，再让奖励模型打一次分，对 7B 以下模型而言，这在现代 GPU 上通常不超过 10 秒。其二，环境是确定性的。给定状态 s_t 和动作 a_t ，下一个状态 s_{t+1} 是完全确定的；奖励函数也是一个神经网络，因此对于同一输入它同样是确定的。虽然 agent 本身按随机策略采样回答，但这种随机性属于策略而不是环境。其三，奖励位于轨迹级别。由于中间 token 奖

励都为 0，只在结束时给出最终分数，因此 RLHF 更接近“单阶段优化”问题，PPO 中依赖价值函数和 TD 学习的机制在这里并没有那么必要。

据此，作者认为，RLHF 中的长期回报可以在计算效率和样本效率上同时做到“不借助价值模型也能高质量获得”。因此，PPO 的价值模型设计并不贴合 RLHF 本身的结构。基于这一认识，论文在下一节提出 ReMax。

4 提出的方法

4.1 REINFORCE 简要回顾

在介绍 ReMax 之前，作者先回顾经典的 REINFORCE 算法，并说明其在 RLHF 场景下如何直接作用于式 (1)。对固定 prompt x ，REINFORCE 使用如下策略梯度：

$$\nabla_{\theta} \mathbb{E}_{a_{1:T} \sim \pi_{\theta}} [r(x, a_{1:T})] = \mathbb{E}_{a_{1:T} \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | x, a_{1:t-1}) r(x, a_{1:T}) \right].$$

这里最后一步使用了经典的 log-derivative trick，其核心思想是交换期望与梯度的顺序，从而得到一个容易采样估计的随机梯度。

定义 score function 为

$$s_{\theta}(x, a_{1:t}) = \nabla_{\theta} \log \pi_{\theta}(a_t | x, a_{1:t-1}). \quad (6)$$

那么给定 N 个 prompt $(x^1, \dots, x^N)$ ，就得到如下随机策略梯度估计：

$$\hat{g}_b(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T s_{\theta}(x^i, a_{1:t}^i) r(x^i, a_{1:T}^i), \quad (7)$$

其中每个 $a_t^i \sim \pi_{\theta}(\cdot | x^i, a_{1:t-1}^i)$ 。于是可以执行梯度上升更新 $\theta_{k+1} \leftarrow \theta_k + \eta_k \hat{g}_b(\theta_k)$ 。从优化角度看，这实际上等价于“用奖励作为权重去做似然最大化”，不同之处在于这些样本来自当前策略 rollout，而不是像监督学习那样提前给定最优标签并固定样本分布。

作者指出，REINFORCE 在 RLHF 中看起来天然合适：它只需要语言模型和奖励模型各查询一次就能估计梯度，而且不需要训练价值模型，因此计算上十分高效。但它仍没有彻底解决 PPO 的问题，因为其随机梯度方差很大。论文在图 4 中给出了直观证据：REINFORCE 的梯度范数剧烈波动，性能也明显较差。

作者进一步解释了其原因。一般来说，REINFORCE 的梯度方差来源于两类随机性：一类是 MDP 环境本身转移的随机性，另一类是策略内部决策的随机性。在 LLM 的 RLHF 中，前者由于环境转移和奖励函数都确定，实际上已经消失；因此问题的根源主要来自后者，即语言模型生成 token 时的内部随机性。当模型规模较小时，REINFORCE 往往还能工作；但扩展到大语言模型时，这种内部随机性会变得非常严重。

此外，作者还指出，REINFORCE 的梯度方差会随着奖励尺度增大而增大，而不同 prompt 的奖励尺度差异往往非常明显。例如，开放式问题和封闭式问题的奖励分布就可能非常不一样。哪怕训练了一个完整 epoch，这种差异仍然存在。相比之下，监督微调相当于固定使用 $r(x, a_{1:T}) = 1$ ，训练要稳定得多。ReMax 的核心目标之一，正是解决这种“不同 prompt 之间以及训练过程中奖励尺度显著变化”所带来的问题。

4.2 从 REINFORCE 到 ReMax

ReMax 受到带 baseline 的 REINFORCE 启发。作者将梯度估计修改为：

$$\hat{g}_e(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T s_\theta(x^i, a_{1:t}^i) (r(x^i, a_{1:T}^i) - b_\theta(x^i)), \quad (8)$$

其中 baseline 取为该 prompt 下模型贪心生成回答的奖励值：

$$b_\theta(x^i) = r(x^i, \bar{a}_{1:T}^i), \quad \bar{a}_t^i \in \arg \max \pi_\theta(\cdot | x^i, \bar{a}_{1:t-1}^i). \quad (9)$$

换言之，对同一个 prompt，ReMax 会同时得到一个随机采样回答和一个贪心回答，然后用二者的奖励差值作为训练信号。之所以命名为 ReMax，是因为它建立在 REINFORCE 之上，并使用了 argmax 对应的贪心基线。

这一做法的直观解释是：贪心回答的奖励值相当于一种针对 prompt 的“归一化参考点”，它把随机回答的得分放在一个更有意义的坐标系里衡量。这样既能够减少梯度估计的方差，也能更好地平衡不同 prompt 之间的奖励尺度差异。作者随后从理论上证明，尽管引入了 baseline，这个梯度估计仍然是无偏的。

与原始 REINFORCE 相比，ReMax 保留了其计算高效的优点，但显著提升了有效性。作者认为，REINFORCE 之所以效果差，很大程度上是因为它不能在训练过程中持续、自适应地平衡不同 prompt 的奖励权重；而 ReMax 使用的贪心 baseline 同时适配 prompt 本身和当前训练状态，因此解决了这一问题。PPO 也能做到类似效果，但代价是必须引入很重的价值模型。

与 DPO 相比，作者强调 ReMax 与其本质上是完全不同的。第一，ReMax 是一种通用强化学习方法，而 DPO 是专门针对“带 KL 正则的偏好学习问题”推导出来的，它依赖 Bradley-Terry-Luce 模型和特定的 KL 正则假设。ReMax 则不受这些假设限制，因此适用范围更广。实践中，这意味着 ReMax 可以使用任意奖励模型，在任意 prompt 数据集上训练，即便这些数据并不包含成对偏好标注；而 DPO 做不到这一点。

DPO 的目标可写为

$$- \sum_{x, a_{1:T}, a'_{1:T}} \log \sigma \left(\beta \log \frac{\pi_\theta(a_{1:T}|x)}{\pi_{\text{REF}}(a_{1:T}|x)} - \beta \log \frac{\pi_\theta(a'_{1:T}|x)}{\pi_{\text{REF}}(a'_{1:T}|x)} \right). \quad (10)$$

第二，ReMax 采用在线学习范式，即不断从当前模型直接采样回答并在线更新；而 DPO 依赖的是静态偏好数据集，其回答在收集后就不再变化，因此会面临 out-of-distribution 的泛化问题。实验中，作者观察到 DPO 的训练质量可能不如 ReMax。第三，DPO 学到的是一种隐式奖励，其形式与 $\log \pi_\theta(a_{1:T}|x) - \log \pi_{\text{REF}}(a_{1:T}|x)$ 成正比。可以把它看作一种固定的 baseline，它能适应 prompt，但不能随着优化过程自适应变化，而 ReMax 的贪心 baseline 则具备这种能力。

最后，作者指出，ReMax 与 DPO 在计算效率上是同量级的。原因在于奖励模型训练完成后只需推理，GPU 资源占用很小；而 ReMax 的在线生成若结合 vLLM 或 DeepSpeed 的混合引擎，也可以做得很快。

4.3 ReMax 的理论分析

为了说明算法设计是有理论依据的，作者首先证明了 ReMax 的梯度估计保持无偏。

命题 1. 由式 (8) 和式 (9) 定义的梯度估计对于目标函数式 (1) 是无偏的，即

$$\mathbb{E}[\hat{g}_e(\theta)] = \nabla_{\theta} \mathbb{E}_{x \sim \rho} \mathbb{E}_{a_{1:T} \sim \pi_{\theta}} [r(x, a_{1:T})].$$

同时，其方差有上界

$$\text{Var}[\hat{g}_e(\theta)] \leq c r_{\max}^2 T^2 S^2 / N,$$

其中 c 为常数， S 是 $\|\nabla_{\theta} \log \pi_{\theta}(a_t | x, a_{1:t-1})\|$ 的上界， $r_{\max} = \max_{x, a_{1:T}} |r(x, a_{1:T})|$ 。

原文附录给出了完整证明。其无偏性只要求 baseline 与随机采样的响应在统计上独立即可，而 ReMax 中通过贪心生成得到的 baseline 正好满足这一点，且实现非常直接。

由于式 (1) 是非凸优化问题，作者进一步给出收敛到局部驻点的结论。

命题 2 (非正式) . 当学习率取 $\eta_k = O(1/\sqrt{k})$ 时，ReMax 在期望意义下收敛到一个驻点。

作者强调，这说明 ReMax 并非一个纯启发式技巧，而是有原则支撑的优化方法。

为了分析其方差缩减性质，论文将问题限制到一个双臂 bandit 场景。虽然这一定理场景很简单，但作者指出，自 Dayan 以来，对于“实际可用的 baseline 值”的方差分析并没有太多实质进展，因此从该例子中获得启发是合理的。

命题 3. 对任意双臂 bandit，若采用 softmax 参数化，且不失一般性地设 a_1 为最优动作、奖励为正，则在最优动作尚未占据主导时，ReMax 的方差小于 REINFORCE 的方差。特别地，当 $\pi_{\theta}(a_1 | x) \leq 0.5$ 时，ReMax 具有明确的方差缩减效果。

这一结论说明：当模型尚未过度自信时，ReMax 更能有效降低方差；当策略已经高度偏向最优动作时，它在最坏情况下也可能略微增加方差。但作者随即指出三点。第一，即使在最坏情况下，ReMax 的方差仍然有界，因此收敛结论不受影响。第二，这一缺点并非 ReMax 独有，即便使用“期望奖励”作为 baseline，也存在类似问题。第三，在 RLHF 中，过度优化本来就不是目标，因为过拟合和 reward overoptimization 反而会带来问题。因此，在“已经优化得过头”的区域里收敛稍慢，实际上未必是坏事。

5 实验

实验部分旨在验证 ReMax 在 RLHF 中相对于现有方法的优势。作者采用 DeepSpeed-Chat 所使用的分布式训练框架，并将实验分为两部分：第一部分使用 Llama-2-7B 与 full-hh-rlhf 数据集，完整实现 RLHF 的三个步骤，从而考察方法的有效性与效率；第二部分则将 ReMax 用于更接近实际应用的场景，使用 Mistral-7B 的 SFT 模型和 UltraRM-13B 奖励模型，重点比较经过 ReMax 训练后的模型与其他开源及闭源模型之间的差距。

5.1 第一部分：ReMax 的有效性

在 full-hh-rlhf 数据集上，作者遵循 InstructGPT 的做法，将数据划分为 20% 用于 SFT，40% 用于奖励模型训练，其余 40% 用于 RL 优化。比较方法包括 SFT、PPO 以及 DPO。全部实验在 4 张 A800-80GB GPU 上进行，采用 bf16 训练。

实验曲线表明，ReMax 获得了与 PPO 相当的奖励水平；同时，从梯度范数来看，ReMax 训练过程稳定，没有出现 RL 算法中常见的大幅震荡。一个有趣现象是，PPO 和 ReMax 的梯度范数都显著低于 DPO。作者认为可能有两点原因：一是 DPO 的梯度估计中需要计算两个不同的 score function，而 ReMax 只需要一个；二是 DPO 无法像 ReMax 那样在 token 维度上做对数似然归一化，否则其理论建模会被破坏，而 ReMax 则可以通过算法第 7 行的形式实现这种归一化，从而稳定训练。

作者还使用 AlpacaEval 数据集上的 805 个多样化 prompt 来评估生成质量。由于这些题目没有人工参考答案，论文采用 GPT-4 作为裁判，比较不同方法相对于 SFT 基线的胜率。结果表明，在相同初始化下，ReMax 带来的提升最大，相比 SFT 增加了 31.4 个百分点。作者还考察了一个更强的组合方案：先用 DPO 训练出一个模型，再以它为初始化，使用 ReMax 在只有 prompt 的数据上继续做奖励优化，即 DPO+ReMax。这个组合取得了 84.7% 的最高胜率。这说明 DPO 可以作为很好的初始化，而 ReMax 能通过在线学习进一步修复 DPO 的分布外偏移问题。

5.2 第一部分：ReMax 的效率

显存方面，ReMax 通过去掉价值模型训练，大约能节省一半的显存。这带来两个直接收益：其一，在 GPU 显存受限时，ReMax 能训练而 PPO 不一定能训练；其二，节省下来的显存可以换成更大的 batch size，从而提升吞吐率，加快整体训练。

作者在 Llama-2-7B 上的实测结果表明，不使用 optimizer offloading 时，PPO 根本无法训练，而 ReMax 可以支持总 batch size 为 96；在开启 offloading 的情况下，PPO 能支持总 batch size 为 112，而 ReMax 能达到 152，大约是前者的 1.4 倍。

训练速度主要取决于 batch size 以及单步迭代耗时。作者进一步将单步时间分成生成时间 T_G 与反向传播时间 T_B 两部分，并用如下近似：

$$\text{PPO} : T_G + T_B = t_{\text{gene}} + 2t_{\text{back}},$$

$$\text{ReMax} : T_G + T_B = 2t_{\text{gene}} + t_{\text{back}}.$$

PPO 需要生成一次响应，同时训练语言模型和价值模型；ReMax 则需要生成两次响应，但只训练语言模型。实验发现，生成时间通常比一次完整反向传播更短，因此总体上 ReMax 更快。在 4 张 GPU 的设置下，ReMax 训练 1 个 epoch 约需 1.8 小时，而 PPO 为 2.9 小时，速度提升约为 1.6 倍。作者还指出，在内存读取速度较慢的机器上，这种优势会更加明显。

进一步地，作者比较了 ReMax 与 DPO 的效率。以 4 张 GPU、不使用 offloading 为例，DPO 可支持的最大 batch size 与 ReMax 相同，都是 96；但由于 DPO 不需要在线采样，其 1 个 epoch 的训练时间为 1.4 小时，而 ReMax 为其约 1.3 倍。作者认为，ReMax 在线采样的开销未来还可以通过 lookahead decoding 与截断式采样进一步减少。

5.3 第二部分：ReMax 在榜单上的表现

为了推进 ReMax 的实际应用，作者采用开源 SFT 模型 Mistral-7B-Instruct-v0.2，并使用开源奖励模型 UltraRM-13B。这种使用“外部训练好的奖励模型”继续优化开源 SFT 模型的方式，本质上是一种弱监督学习，而像 DPO 这样的偏好学习算法并不能直接处理这种设定。

作者进一步提出一个实际问题：到底应该选用什么样的 prompt 数据来做 ReMax 优化？为此，论文考察了三类数据源：其一是 ultrafeedback，其中的 prompt 与奖励模型训练数据分布较接近；其二是 lmsys-chat-1m，包含真实用户对话 prompt；其三是高质量指令数据集 sharegpt-en。模型表现通过 AlpacaEval 和 MT-Bench 两个基准衡量，前者评估相对 text-davinci-003 的胜率，后者评估多轮对话能力，均由 GPT-4 打分。

实验结果有两个主要结论。第一，几乎所有数据源和数据规模下，ReMax 都能带来稳定增益。第二，当数据规模增加到 40k 时，作者观察到明显的 over-optimization，表现反而下降，这说明如何对 prompt 进行合理选择和正则化仍是开放问题。

在这些实验中，sharegpt-en 的 20k prompt 取得了最佳结果：在 AlpacaEval 上达到 94.78% 的胜率，在 MT-Bench 上达到 7.739。这一结果不仅超过了 Llama-2-70B-Chat，也超过了许多强开源 7B 模型，并把开源 7B 模型的性能推到了一个新的高度。作者同时指出，PPO 也许能达到接近效果，但它需要更多的超参数调节与更高的计算资源，这已经超出了他们当前的实验条件。

6 结论

本文指出，LLM 中的 RLHF 与一般强化学习任务在问题结构上存在根本差异，而这种差异会直接影响算法设计。基于这一认识，作者认为 REINFORCE 在计算层面是很合适的起点；但若扩展到大规模模型，就必须解决其随机梯度方差过大的问题。为此，本文提出 ReMax，通过引入贪心基线实现方差缩减。相较于当前该领域最常使用的 PPO，ReMax 具有实现简单、显存占用低、超参数更少、训练更快、任务表现更好的综合优势。总体而言，ReMax 使得在大语言模型上应用 RLHF 变得更加容易、也更加经济。

附录 A：补充相关工作

作者首先说明，将文本生成建模为 MDP 并不是本文首创，早期在神经机器翻译和对话生成中已经出现过。REINFORCE 也曾是一些小规模语言模型上被使用。本文的贡献在于说明：如果能处理好 LLM 决策时内部随机性导致的高方差，那么 REINFORCE 风格的方法同样可以在大模型上成功。

从理论角度看，REINFORCE 的研究历史悠久，但在深度 RL 社区中并不流行，因为在具有随机环境转移和稠密奖励的一般 RL 任务中，它通常不如 actor-critic 方法有效。后者通过价值函数降方差，并借助 TD 学习利用动态规划思想。本文则指出，在 RLHF 这种确定环境、轨迹级奖励、仿真快速的设定下，REINFORCE 反而从计算角度更有吸引力。

关于 baseline 的理论，作者回顾了若干重要工作：Dayan 首先指出，直接使用期望奖励作为 baseline 并不能在全局上保证方差下降；Weaver 与 Tao 以及 Greensmith 等人进一步系统研究了在线学习设定下 baseline 的作用；后来也有工作分析了 REINFORCE 的 regret 或多臂 bandit 下的优化几何。相比之下，ReMax 的 baseline 具有“易计算、可在线更新、与当前策略耦合”的特点。

作者还专门说明，ReMax 并不试图在所有强化学习问题中替代 PPO。在一般 RL 环境中，价值模型仍可能是必要的。此外，已有工作中常见的 best-of-n 采样方法虽然也有效，但会显著增加推理阶段的计算成本，而这在很多应用场景下是难以接受的。ReMax 的出发点不是提升推理时 reranking，而是直接让训练阶段的优化过程更简单有效。

论文还从控制变量 (control variate) 技术的角度解释 ReMax。传统控制变量需要构造一个与目标变量高度相关、且其期望已知的随机变量，用来降低估计方差；而 ReMax 中使用的 $\log \pi_{\theta}(a_{1:T}|x) \cdot b(x)$ 结构则非常容易计算，因此在实践中更可行。

最后，作者坦言，ReMax 并没有解决 RLHF 的所有问题。除了计算效率，如何更好地从人类偏好中拟合奖励函数、如何减轻 reward bias、如何处理 reward gaming 等，依然是重要研究方向。

附录 B：如何处理 KL 正则

由于奖励模型并不完美，单纯最大化其分数可能会导致模型偏离期望行为。为此，实践中通常会在可训练语言模型与一个固定参考模型之间加入 KL 正则。其目标可以写成：

$$\max_{\theta} \mathbb{E}_{x \sim \rho, a_{1:T} \sim \pi_{\theta}} [r(x, a_{1:T})] - \beta \mathbb{E}_{x \sim \rho, a_{1:T} \sim \pi_{\theta}} \left[\log \frac{\pi_{\theta}(a_{1:T}|x)}{\pi_{\text{REF}}(a_{1:T}|x)} \right].$$

这里 $\beta > 0$ 控制 KL 惩罚的强度。

作者给出了两种把 KL 融入 ReMax 的方式。第一种是一阶 KL (one-step KL)：

$$r_e(x, a_{1:t}) = r(x, a_{1:T}) - \beta (\log \pi_{\theta}(a_t|x, a_{1:t-1}) - \log \pi_{\text{REF}}(a_t|x, a_{1:t-1})). \quad (11)$$

第二种是全程 KL (full-step KL)：

$$r_e(x, a_{1:t}) = r(x, a_{1:T}) - \beta \sum_{h=t}^T (\log \pi_{\theta}(a_h|x, a_{1:h-1}) - \log \pi_{\text{REF}}(a_h|x, a_{1:h-1})). \quad (12)$$

后一种形式借鉴了动态规划中的 cost-to-go 思想，也是 PPO 中常用的做法。作者指出，全程 KL 的正则能力更强，但同时会为 KL 估计引入更多随机噪声，因此也更难优化；一阶 KL 更简单，但正则力度相对弱一些。

在 ReMax 中，只需把算法损失中的原始奖励替换成 $r_e(x, a_{1:t})$ 即可。与此同时，原始奖励 $r(x, a_{1:T})$ 仍会再减去 baseline 值 $b(x)$ 。

附录 C：理论证明内容概述

附录 C 给出了命题 1、命题 2 与命题 3 的详细证明。其核心结论包括：

第一，ReMax 的无偏性证明利用了 Bartlett identity。本质上，只要 baseline 是在当前 prompt 下通过与随机采样独立的方式构造出来的，那么与 baseline 相关的那一项在求期望时就会消失。由于 ReMax 的 baseline 由贪心生成得到，而不是从随机采样响应中复用，因此这一条件成立。

第二，方差上界的推导将整体梯度写成独立样本梯度的平均，再利用奖励有界性、score function 有界性以及三角不等式给出上界，从而得到方差随 batch size N 增大而按 $1/N$ 缩小的结论。

第三，关于收敛性，作者在 softmax 自回归参数化下证明目标函数是光滑的，并结合随机梯度下降在非凸优化中的标准分析，得到学习率取 $\eta_k = 1/\sqrt{k}$ 时，最小梯度范数期望值按 $1/\sqrt{K}$ 收敛到 0 的结论，即

$$\min_{1 \leq k \leq K} \mathbb{E}[\|\nabla R(\theta_k)\|^2] \leq \left(r_{\max} + \frac{24r_{\max}^2 T^2 \ln K}{N} \right) \frac{1}{\sqrt{K}}.$$

第四，在双臂 bandit 场景下，作者显式比较了 ReMax 与普通 REINFORCE 的梯度方差。当最优动作概率还没有占据绝对主导时，ReMax 会带来严格的方差缩减；当策略已经极度偏向最优动作时，ReMax 不再保证全局变好，但其最坏情况仍然可控。

此外，附录还给出了“理论上最优 baseline”的表达式：

$$b_{\theta}^*(x) = \frac{\mathbb{E}_{a_{1:T} \sim \pi_{\theta}} [\|\sum_{t=1}^T s_{\theta}(x, a_{1:t})\|^2 r(x, a_{1:T})]}{\mathbb{E}_{a_{1:T} \sim \pi_{\theta}} [\|\sum_{t=1}^T s_{\theta}(x, a_{1:t})\|^2]}.$$

但作者同时指出，这个最优 baseline 在计算上极其不实用，因为它需要对高维梯度项做期望，因此只具有理论意义。

附录 D：技术引理概述

附录 D 主要为收敛性证明服务。作者在自回归 softmax 参数化下证明了目标函数关于参数是 6-smooth 的，即对任意 θ, θ' ，都有

$$\|\nabla R(\theta) - \nabla R(\theta')\|_2 \leq 6\|\theta - \theta'\|_2.$$

为完成这一证明，附录进一步给出了 softmax 概率关于参数的一阶、二阶导表达式，并对 Hessian 的谱范数进行了控制。这些引理本身并不依赖 RLHF 的特殊设定，而更像是对模型参数化形式的标准数学分析。

附录 E：实验细节

在实现层面，作者相对于 DeepSpeed-Chat 原始流程做了几项关键修改。首先，数据处理时不再把 `<|endoftext|>` 作为 padding token，而是将其设为 EOS token，以保证模型能够正确停止生成。其次，在 SFT 阶段，只在 response 部分计算 next-token prediction loss，而不对 prompt 部分计算。第三，在奖励模型训练中，只使用最后一个隐藏状态计算二分类损失，而不是像某些实现那样对所有隐藏状态都施加损失，这样更符合 Bradley-Terry-Luce 理论。

所有主要实验均在四张 A800-80GB GPU 上完成，机器配备 16 个 AMD EPYC 7763 64 核处理器，CPU 内存为 1024GB。Llama-2-7B 的实验采用三个随机种子，作者观察到结果差异并不显著，这在大规模神经网络中较常见。为了减少计算负担，Mistral-7B 实验只使用单个随机种子。

对于 Llama-2-7B，SFT 阶段每卡 batch size 为 30，学习率为 10^{-5} ，使用 cosine schedule，训练两个 epoch；奖励模型阶段每卡 batch size 为 36，学习率同样为 10^{-5} ，带有 0.1 的 weight decay，训练两个 epoch，最终评估准确率约为 63%；RL 阶段学习率降为 10^{-6} ，训练一个 epoch，PPO 与 ReMax 的 KL 系数都设为 0.1，生成时温度设为 1，top-p 为 0.9，而 DPO 的 β 在 $\{0.01, 0.05, 0.1\}$ 中调优后取 0.05。

对于 Mistral-7B，作者使用的是 Mistral-7B-Instruct-v0.2 和 UltraRM-13B。由于 DeepSpeed 当前并不很好支持 Mistral 的架构，混合训练引擎无法启用，参考模型只能用 ZeRO-0，LLM 训练采用 ZeRO-2 配合 optimizer state offloading，奖励模型则用 ZeRO-3 和 parameter offloading。为降低成本，作者仅保留长度不超过 384 的 prompt，同时把 response 最大长度设为 384，总长度上限为 784。训练中采用学习率 5×10^{-7} 、reward clipping 为 1.0、温度 0.7、top-p 0.9，并使用全程 KL 正则。由于训练成本太高，这些超参数没有被细调，作者认为后续若进一步调参，性能还可能继续提升。

附录 E 还详细解释了图 2 和表 2 的显存、时间统计方法。以 Llama-2-7B 为例，若采用 bfloat16，模型参数本身约占 12.55GB；优化器状态采用 float32，需要约 75.31GB；当序列长度为 512、batch size 为 8 时，梯度和激活分别需要 12.55GB 与 45.61GB。因此，一个可训练的 Llama-2-7B 模型总共约需 147.02GB 显存，而一个仅推理使用的模型则约需 12.55GB。由此可得，PPO 需要两个可训练模型（LLM 和 value model）加两个仅推理模型（reward model 和 reference model），总计约 319.14GB；而 ReMax 只需一个可训练模型加两个推理模型，总计约 172.12GB，也就是 PPO 的 54%。

附录 F：补充结果

附录 F 首先展示了 REINFORCE 的训练不稳定性。作者在 OPT-1.3B 和 Llama-2-7B 上都观察到，REINFORCE 的梯度范数明显更大、波动更剧烈，评估奖励、困惑度以及 KL 指标也更差。模型虽不一定总是数值发散，但优化环境显然更加困难，因此作者在主实验中放弃了对 REINFORCE 的进一步研究，转而聚焦于带方差缩减的 ReMax。

接着，作者研究了 ReMax 中两种 KL 正则形式的消融。结果表明，一阶 KL 和全程 KL 都能发挥作用，但全程 KL 的正则效力更强，因此为了得到可比的效果，需要把其系数从 0.1 降到 0.01。无论使用哪一种，关键都在于对 β 做合适调节。

附录 F.3 提出了一个非常直接的加速版本 ReMax-fast。作者观察到，在很多情况下，贪心 baseline 的奖励只用一段较短的截断回答来近似就已经足够。例如，把贪心生成长度减半，依然能得到与完整回答高度相关的奖励值。因此，ReMax-fast 在生成贪心 baseline 时限制最大长度，从而减少生成时间。作者实验发现，将贪心 baseline 长度从 256 减到 128 时，性能几乎不受影响；若进一步减到 64，则会明显损害性能。正常采样回答的生成长度则保持不变。

由于 transformer 自注意力的复杂度近似是二次的，把 greedy response 长度减半理论上能将该部分时间减少到原来的约 75%。实验中，ReMax 相比 PPO 的加速比由 1.6 倍提升到约 2.1 倍。

附录 F.4 还展示了 ReMax 在传统 NLP 任务中的适用性。作者在一个 137M 参数的 GPT-2 上做 IMDB 影评续写实验，奖励由情感分类器提供，目标是让模型继续生成 48 个 token，并保持文本正向情感与流畅性。结果表明，ReMax 同样能够有效最大化奖励，相比 PPO 取得了约 2.2 倍的加速，同时显存占用也明显更低。

附录 G：响应样例说明

附录 G 展示了多组模型输出样例，包括分布外测试样例和训练分布内样例。总体观察是，ReMax 相比 SFT、DPO 和 PPO，往往能生成信息更充实、解释更完整、价值观更稳健的回答；在涉及安全、偏见与伦理的 prompt 上，ReMax 也表现出更稳妥的行为倾向。原论文在这一部分给出了完整英文样例，篇幅较长，因此本翻译稿不逐条重排模型输出正文，仅保留该附录的结论性说明。

译者补充小结

从大模型后训练的角度看，这篇论文最值得注意的地方，不仅在于它提出了一个“比 PPO 更轻”的 RLHF 算法，更在于它重新审视了 RLHF 的问题结构本身。作者并不是简单地把 PPO 做工程瘦身，而是先指出 RLHF 在 LLM 中具有“确定环境、轨迹级奖励、快速 rollout”这三个性质，再据此说明为什么价值模型并非必要，最后才自然地落到 ReMax 这一设计上。也正因为如此，ReMax 的说服力同时来自理论、工程和实验三个层面。

原文来源：用户上传文件 ReMax.pdf。